

Transformers

A from-basics-to-interview-depth guide for ML placements, AI engineering roles, research internships, and Transformer-based projects.

1. Overview

A **Transformer** is a neural-network architecture for processing sequences and other structured collections of tokens. It was introduced in the 2017 paper *Attention Is All You Need*. Unlike recurrent neural networks (RNNs), a Transformer does not have to process tokens strictly one after another. Its central operation, **attention**, lets each token directly gather information from other relevant tokens.

For a sentence such as:

The animal did not cross the street because **it** was tired.

the representation of **it** can attend strongly to **animal**, even though other words occur between them. This direct interaction makes long-range relationships easier to learn than in a plain RNN.

Transformers are useful because they:

- model relationships between distant tokens;
- train efficiently on GPUs through parallel matrix operations;
- scale predictably with more data, parameters, and compute;
- support pretraining followed by fine-tuning, prompting, retrieval, or adapters;
- work beyond text when images, audio, video, molecules, or actions are converted into tokens.

Major real-world uses include:

- machine translation, summarization, search, question answering, and classification;
- large language models such as GPT-style and Llama-style models;
- bidirectional language encoders such as BERT;
- vision models such as Vision Transformer (ViT) and DETR;
- speech recognition and multimodal assistants;
- embedding models, rerankers, RAG systems, code assistants, and autonomous agents;
- protein, molecule, time-series, recommendation, and reinforcement-learning models.

The word **Transformer** describes an architectural family, not one specific model. BERT, T5, GPT, ViT, and modern decoder-only LLMs use different subsets or arrangements of the same fundamental components.

2. Intuition

2.1 Attention as a searchable notebook

Imagine that every word in a sentence has three notes:

- a **query**: what information am I looking for?
- a **key**: what kind of information do I contain?

- a **value**: what information should I provide if selected?

For each token, the Transformer compares its query with every allowed key. A high query-key similarity produces a high attention weight. The token then takes a weighted mixture of the corresponding values.

For example, in “The cat drank the milk because **it** was thirsty,” the query generated for **it** may match the key for **cat** more than the key for **milk**. Attention therefore moves more information from **cat** into the contextual representation of **it**.

2.2 Why several attention heads?

One relationship is not enough. A word may simultaneously depend on:

- its grammatical subject;
- a previous name;
- nearby punctuation;
- the broader topic;
- positional or formatting clues.

Multi-head attention gives the model several learned attention spaces. Different heads are free to capture different patterns, although individual heads are not guaranteed to have a clean human-readable meaning.

2.3 The full block

An attention layer lets tokens exchange information. A small feed-forward network then transforms each token independently. Residual connections preserve the old representation, and layer normalization keeps optimization stable. Stacking these blocks repeatedly creates increasingly contextual representations.

2.4 Encoder, decoder, and decoder-only intuition

- An **encoder** reads all input tokens together and builds contextual representations. This is useful for understanding or classifying an input.
- An **encoder-decoder** first understands the source and then generates a target while consulting the source. This suits translation and summarization.
- A **decoder-only** model predicts the next token from previous tokens. Repeating this prediction produces text. Most general-purpose LLMs use this design.

3. Prerequisites

Mathematics

- Vectors, matrices, matrix multiplication, transpose, and dot products
- Basic probability and conditional probability
- Softmax, logarithms, and cross-entropy
- Derivatives, gradients, and the chain rule
- Basic optimization: gradient descent, Adam, and learning rates
- Familiarity with big-O notation

Deep learning

- Linear layers and learned parameters
- Embedding tables
- Activation functions such as ReLU, GELU, and SiLU
- Backpropagation and mini-batch training
- Dropout, residual connections, and normalization
- Train, validation, and test splits

NLP and programming

- Tokenization and vocabulary construction
- Padding, sequence length, and batching
- Python, NumPy, and PyTorch tensors
- Helpful but not mandatory: RNNs, CNNs, transfer learning, and Hugging Face

Recommended shape notation

Symbol	Meaning
B	batch size
T or n	sequence length / number of tokens
d_{model}	model or embedding width
h	number of attention heads
d_k, d_v	key/query and value width per head
d_{ff}	hidden width of the feed-forward network
V	vocabulary size
L	number of Transformer blocks

4. Core Concepts

4.1 Tokens and tokenization

What it means: Raw text is split into discrete token IDs. Modern tokenizers commonly use subword units learned by BPE, WordPiece, or Unigram algorithms.

Why it matters: The network accepts integers from a finite vocabulary, not raw strings. Tokenization determines sequence length, handling of rare words, multilingual coverage, and part of the model's compute cost.

Example: `unbelievable` might become `un`, `believ`, `able`; an uncommon word can therefore be represented without a unique vocabulary entry.

Common interview angle: Why not tokenize only by word or character? Word vocabularies are huge and suffer from unknown words; character sequences are universal but long. Subwords are a practical compromise.

4.2 Token embeddings

What it means: A learned table $E \in \mathbb{R}^{V \times d_{model}}$ maps each token ID to a dense vector.

Why it matters: Attention operates on continuous vectors. During training, tokens used in similar contexts often develop related representations.

Example: Looking up token ID 42 returns row E_{42} .

Common interview angle: An embedding is initially context-free. The same token begins with the same vector in every sentence; stacked Transformer layers make it contextual.

4.3 Positional information

What it means: Self-attention alone is permutation equivariant: without position information, reordering tokens reorders outputs but does not tell the model which order is correct. The model therefore receives absolute or relative position information.

Why it matters: "dog bites man" and "man bites dog" contain the same tokens but have different meanings.

Example: The input may be $X = E_{token} + E_{position}$, or query/key vectors may be rotated using RoPE.

Common interview angle: Sinusoidal embeddings are fixed and can extrapolate imperfectly; learned absolute embeddings are flexible but tied to trained positions; relative methods encode pairwise distance and often handle long contexts better.

4.4 Query, key, and value projections

What it means: For input X , learned matrices create:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V.$$

Why it matters: The model needs separate spaces for matching information (query/key) and transporting information (value).

Example: A token's query may request "which noun controls me?", while another token's key signals "I am a candidate noun." Its value carries the information to transfer.

Common interview angle: Q, K, and V usually come from the same sequence in self-attention and from different sequences in cross-attention.

4.5 Scaled dot-product attention

What it means: Attention compares every query with allowed keys, normalizes scores, and mixes values:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} + M \right) V.$$

Why it matters: It provides content-dependent information routing. The output for a token changes according to its context.

Example: If a query has normalized weights $[0.1, 0.7, 0.2]$, its result is $0.1v_1 + 0.7v_2 + 0.2v_3$.

Common interview angle: Dividing by $\sqrt{d_k}$ prevents dot products from growing in variance with dimension and pushing softmax into saturated, low-gradient regions.

4.6 Self-attention and cross-attention

Self-attention: Q, K, and V all originate from the same sequence. Tokens exchange information within that sequence.

Cross-attention: Queries originate from the decoder state, while keys and values originate from encoder outputs or another modality.

Example: During translation, a generated French token can query English encoder representations.

Common interview angle: Cross-attention is the bridge between source and target in an encoder-decoder model. Decoder-only models normally do not have it unless augmented with another input stream.

4.7 Multi-head attention

What it means: Attention is run in parallel across h projected subspaces:

$$\begin{aligned} head_i &= \text{Attention}(QW_i^Q, KW_i^K, VW_i^V), \\ \text{MHA}(Q, K, V) &= \text{Concat}(head_1, \dots, head_h)W_O. \end{aligned}$$

Why it matters: Different representation subspaces can model different relationships. With the usual $d_k = d_{model}/h$, using more heads does not multiply the leading projection cost by h .

Example: One head may emphasize nearby syntax while another tracks long-distance entity information.

Common interview angle: `d_model` must normally be divisible by the number of heads. More heads do not automatically mean a better model; each head becomes narrower.

4.8 Attention masks

What it means: A mask adds a very negative value to forbidden attention logits before softmax.

- **Padding mask:** prevents attention to padding tokens.
- **Causal mask:** prevents position t from looking at future positions $> t$.
- **Structured mask:** enforces local windows, blocks, prefixes, or task-specific connectivity.

Why it matters: A wrong mask can leak labels during training or corrupt padded batches.

Example: For causal attention, row t can attend only to columns $0, \dots, t$.

Common interview angle: Masking after softmax is incorrect because forbidden positions already receive probability mass. Mask logits before softmax.

4.9 Position-wise feed-forward network

What it means: Each token independently passes through the same nonlinear MLP:

$$\text{FFN}(x) = W_2 \sigma(W_1 x + b_1) + b_2.$$

Modern LLMs often use gated forms such as SwiGLU.

Why it matters: Attention mixes information across positions; the FFN performs nonlinear feature transformation within each position. It contains a large fraction of model parameters and computation.

Example: A $d_{model} = 768$ representation may expand to $d_{ff} = 3072$, apply GELU, and project back to 768.

Common interview angle: The same FFN weights are used at every token position, but each Transformer layer has its own FFN.

4.10 Residual connections and LayerNorm

What it means: A sublayer output is added to its input. LayerNorm normalizes features within each token.

Post-norm form:

$$y = \text{LN}(x + \text{Sublayer}(x)).$$

Pre-norm form:

$$y = x + \text{Sublayer}(\text{LN}(x)).$$

Why it matters: Residual paths improve gradient flow. LayerNorm stabilizes activation scales without depending on other batch examples.

Example: If attention initially learns a small correction, the residual path still carries the original representation forward.

Common interview angle: Most modern large models use pre-norm because it tends to optimize deep networks more reliably. LayerNorm, unlike BatchNorm, works naturally with varying sequence lengths and batch sizes.

4.11 Encoder block

An encoder block usually contains:

1. bidirectional self-attention;
2. residual connection and normalization;
3. position-wise FFN;
4. another residual connection and normalization.

Every non-padding input position may attend to every other input position. BERT-style masked-language models use encoder stacks.

Common interview angle: For classification, a special token representation such as [CLS] or a pooled representation is fed to a task head.

4.12 Decoder block

The original encoder-decoder Transformer decoder contains:

1. causal self-attention over generated target tokens;
2. cross-attention to encoder output;

3. an FFN;
4. residual and normalization operations around these sublayers.

A decoder-only LLM removes encoder cross-attention and stacks causal self-attention blocks.

Common interview angle: "Decoder" is overloaded. An original seq2seq decoder includes cross-attention; a GPT-style decoder block generally does not.

4.13 Output projection and language-model head

Final hidden states are projected to vocabulary logits:

$$z_t = h_t W_{vocab} + b, \quad p(x_{t+1} \mid x_{\leq t}) = \text{softmax}(z_t).$$

The output matrix is often **weight-tied** with the input embedding table, reducing parameters and often improving learning.

Common interview angle: Logits are unnormalized real numbers. Softmax converts them into a probability distribution; generation then chooses or samples a token.

4.14 Pretraining objectives

Objective	Prediction	Typical architecture	Examples
Causal language modeling	next token from previous tokens	decoder-only	GPT-style, Llama-style
Masked language modeling	masked tokens from both sides	encoder-only	BERT, RoBERTa
Denoising seq2seq	reconstruct target from corrupted input	encoder-decoder	T5, BART
Contrastive learning	align paired representations	dual encoder / multimodal	CLIP-like models

Common interview angle: Causal LM training is parallel over all target positions because the causal mask prevents leakage, even though autoregressive inference generates sequentially.

4.15 Generation and decoding

- **Greedy decoding:** choose the highest-probability token each step.
- **Beam search:** keep several high-scoring sequences; useful when there is a relatively narrow correct output, such as translation.
- **Temperature:** divide logits by τ ; lower values sharpen, higher values flatten.
- **Top-k:** sample only among the k most likely tokens.
- **Top-p / nucleus:** sample from the smallest set whose cumulative probability reaches p .
- **Repetition or length penalties:** adjust scores to control common decoding failures.

Common interview angle: Decoding strategy changes output behavior but does not change model weights or improve the model's factual knowledge.

4.16 KV cache

During autoregressive inference, previously computed keys and values do not need to be recomputed. A **KV cache** stores them for every layer.

Why it matters: It greatly reduces repeated computation during generation. Its memory grows approximately linearly with batch size, generated/context length, layer count, and KV-head dimension.

Common interview angle: KV caching speeds decoding but does not remove the sequential dependency between newly generated tokens. Multi-query attention and grouped-query attention reduce KV-cache size.

4.17 In-context learning versus parameter learning

The prompt changes the activations used for the current request but does not normally update model parameters. Fine-tuning updates parameters across examples. Retrieval adds external context. These mechanisms are complementary and should not be confused.

5. Algorithm / Working Process

5.1 Encoder forward pass

Suppose the input is a batch of token sequences with shape (B, T) .

1. **Tokenize:** Convert raw inputs into token IDs, padding masks, and possibly segment IDs.
2. **Embed:** Look up token vectors, producing (B, T, d_{model}) .
3. **Add/inject position:** Add learned or sinusoidal positions, or apply a relative/rotary position method inside attention.
4. **Project Q, K, V:** Produce per-head query, key, and value tensors.
5. **Compute attention scores:** Multiply queries by transposed keys and scale by $\sqrt{d_k}$.
6. **Apply padding/structured masks:** Make forbidden logits effectively $-\infty$.
7. **Normalize and aggregate:** Apply softmax and multiply by values.
8. **Combine heads:** Concatenate head outputs and apply an output projection.
9. **Residual and normalization:** Add the attention result to the residual stream.
10. **Apply FFN:** Transform each position with the shared position-wise MLP.
11. **Repeat:** Pass through L blocks.
12. **Task head:** Use token states for token labeling, pooling for classification, or expose all states to a decoder.

5.2 Encoder-decoder forward pass

1. Encode the source sequence bidirectionally.
2. Shift target tokens right so the decoder sees a start token and previous gold tokens.
3. Apply causal self-attention over the shifted target.
4. Apply cross-attention: decoder states are queries; encoder states are keys and values.
5. Apply the decoder FFN and repeat across decoder layers.
6. Project decoder states to vocabulary logits.
7. Compare each position with the next target token using cross-entropy.

5.3 Decoder-only language-model forward pass

Given tokens $[x_0, x_1, \dots, x_{T-1}]$:

1. Feed them into a causal Transformer.
2. Hidden state at position t can use only $x_0, \dots, x_t$.
3. Logits at position t predict x_{t+1} .
4. Compute loss for all valid next-token positions in parallel.

The input-target shift is:

```
input:  [BOS, The, cat, sat]
target: [The, cat, sat, EOS]
```

5.4 Training process

1. Gather and clean task data or pretraining corpora.
2. Fit/select a tokenizer and convert examples to IDs.
3. Form batches, masks, and shifted labels.
4. Run the Transformer forward pass.
5. Compute the objective, usually token-level cross-entropy.
6. Backpropagate gradients.
7. Clip gradients if needed, then update parameters with AdamW or another optimizer.
8. Adjust the learning rate, commonly with warm-up followed by decay.
9. Evaluate held-out loss and downstream/task metrics.
10. Save checkpoints and resume safely after interruptions.

Teacher forcing: During supervised seq2seq training, the decoder receives ground-truth previous target tokens. This permits parallel training but creates a gap from inference, where it receives its own previous predictions.

5.5 Autoregressive inference process

1. Tokenize the prompt.
2. Run a **prefill** pass over the prompt and create the KV cache.
3. Select the next token from final-position logits using a decoding strategy.
4. Append that token.
5. Run one decoding step using cached keys and values.
6. Repeat until an end token, stop sequence, or token limit is reached.
7. Detokenize the generated IDs.

5.6 Classification inference process

1. Tokenize and pad input examples.
2. Run the encoder once.
3. Pool a designated token or valid token states.
4. Apply a linear task head.
5. Use sigmoid for independent labels or softmax/argmax for mutually exclusive classes.

6. Mathematical Foundation

6.1 Embedding and position representation

For token ID x_t at position t :

$$h_t^{(0)} = E[x_t] + P[t].$$

Here E is a learned token embedding and P is a learned or fixed positional representation.

The original sinusoidal encoding uses:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right),$$

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right).$$

Different dimensions oscillate at different frequencies, giving the model information about absolute position and relative displacement.

6.2 Why scaled dot products?

Let components of q and k be independent with mean 0 and variance 1. Then:

$$q \cdot k = \sum_{i=1}^{d_k} q_i k_i$$

has variance approximately d_k . Dividing by $\sqrt{d_k}$ gives variance near 1:

$$\text{Var}\left(\frac{q \cdot k}{\sqrt{d_k}}\right) \approx 1.$$

This keeps logits in a range where softmax gradients are useful.

6.3 Scaled dot-product attention with shapes

For one head:

$$Q \in \mathbb{R}^{B \times T_q \times d_k}, \quad K \in \mathbb{R}^{B \times T_k \times d_k}, \quad V \in \mathbb{R}^{B \times T_k \times d_v}.$$

Scores:

$$S = \frac{QK^\top}{\sqrt{d_k}} \in \mathbb{R}^{B \times T_q \times T_k}.$$

After adding mask M :

$$A_{ij} = \frac{\exp(S_{ij} + M_{ij})}{\sum_{r=1}^{T_k} \exp(S_{ir} + M_{ir})}.$$

Output:

$$O = AV \in \mathbb{R}^{B \times T_q \times d_v}.$$

If position j is forbidden for query i , set $M_{ij} = -\infty$, making $A_{ij} = 0$.

6.4 Multi-head attention dimensions

Commonly:

$$d_k = d_v = \frac{d_{model}}{h}.$$

Each head returns shape (B, T, d_v) . Concatenation restores $(B, T, h d_v) = (B, T, d_{model})$, after which W_O mixes information between heads.

6.5 Feed-forward network and gated variants

Classic FFN:

$$FFN(x) = W_2 \text{GELU}(W_1 x + b_1) + b_2.$$

SwiGLU-like form:

$$SwiGLU(x) = W_2 (\text{SiLU}(x W_g) \odot (x W_u)).$$

The element-wise product $\odot$ lets a learned gate control which features pass through.

6.6 Layer normalization

For one token vector $x \in \mathbb{R}^{d_{model}}$:

$$\mu = \frac{1}{d_{model}} \sum_i x_i, \quad \sigma^2 = \frac{1}{d_{model}} \sum_i (x_i - \mu)^2,$$

$$LN(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta.$$

Normalization occurs across feature dimensions for each token independently. RMSNorm is a common alternative that omits mean centering.

6.7 Causal language-model factorization

The joint probability of a sequence is decomposed by the chain rule:

$$p(x_1, \dots, x_T) = \prod_{t=1}^T p(x_t \mid x_{<t}).$$

The model learns each conditional next-token distribution.

6.8 Cross-entropy and negative log-likelihood

For target token y_t , predicted distribution p_t , and valid-token set $\mathcal{T}$:

$$\mathcal{L}_{CE} = -\frac{1}{|\mathcal{T}|} \sum_{t \in \mathcal{T}} \log p_t(y_t).$$

Padding labels are excluded, commonly with `ignore_index`. Minimizing cross-entropy is equivalent to maximizing the likelihood of the observed target tokens.

With label smoothing ϵ , the target distribution is mixed with a small amount of probability mass over non-target classes. This can reduce overconfidence but changes the interpretation of raw loss and perplexity.

6.9 Perplexity

For average token negative log-likelihood $\mathcal{L}$:

$$PPL = \exp(\mathcal{L}).$$

Intuitively, lower perplexity means the model assigns more probability to the observed sequence. Perplexities are comparable only when tokenization, data, and evaluation procedure are comparable.

6.10 Optimization with AdamW

For gradient g_t , Adam tracks moments:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t,$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2.$$

After bias correction, AdamW applies an adaptive update plus decoupled weight decay. In simplified form:

$$\theta_{t+1} = \theta_t - \text{eta} \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} - \eta \lambda \theta_t.$$

Warm-up avoids large unstable updates before moment estimates and activations have settled. Later decay improves convergence.

6.11 Temperature sampling

For logits z_i and temperature $\tau > 0$:

$$p_i = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}.$$

- $\tau \rightarrow 0$: distribution approaches argmax.
- $\tau = 1$: unchanged model distribution.
- $\tau > 1$: flatter and more random distribution.

Temperature is not a training metric and cannot fix a poorly trained model.

6.12 Parameter-count approximation

Ignoring biases and normalization parameters, one standard block has approximately:

- attention projections: $4d_{model}^2$ for Q, K, V, and output;
- FFN: $2d_{model}d_{ff}$;
- block total: $4d_{model}^2 + 2d_{model}d_{ff}$.

If $d_{ff} \approx 4d_{model}$, this is about $12d_{model}^2$ parameters per block. Embeddings add approximately Vd_{model} , possibly shared with the output head.

7. Practical Implementation

The following is a compact, end-to-end **decoder-only character language model** in PyTorch. It demonstrates embeddings, positions, causal multi-head attention, pre-norm residual blocks, next-token training, evaluation, and generation. A character tokenizer keeps the example self-contained; production NLP normally uses a subword tokenizer.

7.1 From-scratch PyTorch implementation

```
import math
import torch
from torch import nn
from torch.nn import functional as F

class CausalSelfAttention(nn.Module):
    def __init__(self, d_model: int, n_heads: int, dropout: float):
        super().__init__()
        if d_model % n_heads != 0:
            raise ValueError("d_model must be divisible by n_heads")

        self.n_heads = n_heads
        self.head_dim = d_model // n_heads
        self.qkv = nn.Linear(d_model, 3 * d_model)
        self.out = nn.Linear(d_model, d_model)
        self.dropout = dropout

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        batch, time, channels = x.shape

        # (B, T, 3C) -> three tensors of shape (B, heads, T, head_dim)
        q, k, v = self.qkv(x).chunk(3, dim=-1)
        q = q.view(batch, time, self.n_heads, self.head_dim).transpose(1, 2)
        k = k.view(batch, time, self.n_heads, self.head_dim).transpose(1, 2)
        v = v.view(batch, time, self.n_heads, self.head_dim).transpose(1, 2)

        # PyTorch applies scaling, softmax, causal masking, and dropout
        # efficiently.
        attended = F.scaled_dot_product_attention(
            q, k, v,
            attn_mask=None,
            dropout_p=self.dropout if self.training else 0.0,
            is_causal=True,
        )

        # Merge heads: (B, heads, T, head_dim) -> (B, T, C)
        attended = attended.transpose(1, 2).contiguous().view(batch, time,
channels)
```

```

        return self.out(attended)

class FeedForward(nn.Module):
    def __init__(self, d_model: int, dropout: float):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(d_model, 4 * d_model),
            nn.GELU(),
            nn.Linear(4 * d_model, d_model),
            nn.Dropout(dropout),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.net(x)

class TransformerBlock(nn.Module):
    def __init__(self, d_model: int, n_heads: int, dropout: float):
        super().__init__()
        self.norm1 = nn.LayerNorm(d_model)
        self.attention = CausalSelfAttention(d_model, n_heads, dropout)
        self.norm2 = nn.LayerNorm(d_model)
        self.ffn = FeedForward(d_model, dropout)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # Pre-norm residual architecture.
        x = x + self.attention(self.norm1(x))
        x = x + self.ffn(self.norm2(x))
        return x

class TinyTransformerLM(nn.Module):
    def __init__(
        self,
        vocab_size: int,
        context_length: int = 128,
        d_model: int = 128,
        n_heads: int = 4,
        n_layers: int = 4,
        dropout: float = 0.1,
    ):
        super().__init__()
        self.context_length = context_length
        self.token_embedding = nn.Embedding(vocab_size, d_model)
        self.position_embedding = nn.Embedding(context_length, d_model)
        self.blocks = nn.Sequential(*[
            TransformerBlock(d_model, n_heads, dropout)
            for _ in range(n_layers)
        ])
        self.final_norm = nn.LayerNorm(d_model)
        self.lm_head = nn.Linear(d_model, vocab_size, bias=False)

    # Input/output weight tying saves parameters.

```

```

        self.lm_head.weight = self.token_embedding.weight

    def forward(
        self,
        token_ids: torch.Tensor,
        targets: torch.Tensor | None = None,
    ) -> tuple[torch.Tensor, torch.Tensor | None]:
        batch, time = token_ids.shape
        if time > self.context_length:
            raise ValueError(f"Sequence length {time} exceeds context length")

        positions = torch.arange(time, device=token_ids.device)
        x = self.token_embedding(token_ids) + self.position_embedding(positions)
        x = self.blocks(x)
        logits = self.lm_head(self.final_norm(x))

        loss = None
        if targets is not None:
            loss = F.cross_entropy(
                logits.reshape(-1, logits.size(-1)),
                targets.reshape(-1),
            )
        return logits, loss

    @torch.no_grad()
    def generate(
        self,
        token_ids: torch.Tensor,
        max_new_tokens: int,
        temperature: float = 1.0,
        top_k: int | None = None,
    ) -> torch.Tensor:
        if temperature <= 0:
            raise ValueError("temperature must be positive")

        self.eval()
        for _ in range(max_new_tokens):
            # Keep only positions that fit the learned context window.
            context = token_ids[:, -self.context_length:]
            logits, _ = self(context)
            next_logits = logits[:, -1, :] / temperature

            if top_k is not None:
                k = min(top_k, next_logits.size(-1))
                threshold = torch.topk(next_logits, k).values[:, -1, None]
                next_logits = next_logits.masked_fill(next_logits < threshold, -
torch.inf)

            probabilities = F.softmax(next_logits, dim=-1)
            next_token = torch.multinomial(probabilities, num_samples=1)
            token_ids = torch.cat((token_ids, next_token), dim=1)

        return token_ids

```

```

def make_batch(
    data: torch.Tensor,
    batch_size: int,
    context_length: int,
    device: torch.device,
) -> tuple[torch.Tensor, torch.Tensor]:
    """Sample chunks and their one-token-shifted targets."""
    starts = torch.randint(0, len(data) - context_length - 1, (batch_size,))
    x = torch.stack([data[i:i + context_length] for i in starts])
    y = torch.stack([data[i + 1:i + context_length + 1] for i in starts])
    return x.to(device), y.to(device)

@torch.no_grad()
def estimate_loss(
    model: nn.Module,
    data: torch.Tensor,
    batch_size: int,
    context_length: int,
    device: torch.device,
    batches: int = 20,
) -> float:
    model.eval()
    losses = []
    for _ in range(batches):
        x, y = make_batch(data, batch_size, context_length, device)
        _, loss = model(x, y)
        losses.append(loss.item())
    return sum(losses) / len(losses)

def main() -> None:
    torch.manual_seed(42)
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    # Replace this with a larger text file for a meaningful project.
    text = ("transformers learn which previous tokens matter. " * 500)
    vocabulary = sorted(set(text))
    char_to_id = {char: i for i, char in enumerate(vocabulary)}
    id_to_char = {i: char for char, i in char_to_id.items()}

    encode = lambda s: [char_to_id[c] for c in s]
    decode = lambda ids: "".join(id_to_char[i] for i in ids)

    all_ids = torch.tensor(encode(text), dtype=torch.long)
    split = int(0.9 * len(all_ids))
    train_ids, validation_ids = all_ids[:split], all_ids[split:]

    context_length = 64
    batch_size = 32
    model = TinyTransformerLM(
        vocab_size=len(vocabulary),
        context_length=context_length,

```

```

        d_model=128,
        n_heads=4,
        n_layers=3,
    ).to(device)

    optimizer = torch.optim.AdamW(model.parameters(), lr=3e-4, weight_decay=0.01)

    for step in range(501):
        model.train()
        x, y = make_batch(train_ids, batch_size, context_length, device)
        _, loss = model(x, y)

        optimizer.zero_grad(set_to_none=True)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
        optimizer.step()

        if step % 100 == 0:
            validation_loss = estimate_loss(
                model, validation_ids, batch_size, context_length, device
            )
            print(f"step={step:3d} train={loss.item():.3f} val=
{validation_loss:.3f}")

        prompt = torch.tensor([encode("transformers ")], device=device)
        generated = model.generate(prompt, max_new_tokens=100, temperature=0.8,
top_k=8)
        print(decode(generated[0].tolist()))

    if __name__ == "__main__":
        main()

```

7.2 Practical Hugging Face inference

For real applications, use a pretrained model rather than training from scratch:

```

import torch
from transformers import AutoModelForSequenceClassification, AutoTokenizer

model_name = "distilbert-base-uncased-finetuned-sst-2-english"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSequenceClassification.from_pretrained(model_name)
model.eval()

texts = [
    "The interview preparation guide was extremely useful.",
    "The model was slow and the answer was incorrect.",
]

batch = tokenizer(

```

```

        texts,
        padding=True,
        truncation=True,
        return_tensors="pt",
    )

    with torch.no_grad():
        logits = model(**batch).logits
        probabilities = logits.softmax(dim=-1)
        predictions = probabilities.argmax(dim=-1)

    for text, label_id, confidence in zip(
        texts, predictions.tolist(), probabilities.max(dim=-1).values.tolist()
    ):
        label = model.config.id2label[label_id]
        print({"text": text, "label": label, "confidence": round(confidence, 3)})

```

7.3 Minimal fine-tuning skeleton

```

from datasets import load_dataset
from transformers import (
    AutoModelForSequenceClassification,
    AutoTokenizer,
    DataCollatorWithPadding,
    Trainer,
    TrainingArguments,
)

checkpoint = "distilbert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(checkpoint)
dataset = load_dataset("glue", "sst2")

def tokenize(batch):
    return tokenizer(batch["sentence"], truncation=True)

tokenized = dataset.map(tokenize, batched=True)
model = AutoModelForSequenceClassification.from_pretrained(checkpoint,
    num_labels=2)

args = TrainingArguments(
    output_dir="transformer-sst2",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=32,
    num_train_epochs=3,
    weight_decay=0.01,
    eval_strategy="epoch",
    save_strategy="epoch",
    load_best_model_at_end=True,

```

```

        report_to="none",
    )

    trainer = Trainer(
        model=model,
        args=args,
        train_dataset=tokenized["train"],
        eval_dataset=tokenized["validation"],
        processing_class=tokenizer,
        data_collator=DataCollatorWithPadding(tokenizer),
    )
    trainer.train()
    trainer.evaluate()

```

8. Code Explanation

8.1 CausalSelfAttention

The single `qkv` linear layer calculates all three projections efficiently. `chunk(3, dim=-1)` separates them, and `view(...).transpose(1, 2)` changes the representation from `(batch, time, channels)` to `(batch, heads, time, head_dim)`.

`scaled_dot_product_attention` performs the same mathematical operation described earlier. With `is_causal=True`, future positions are hidden. PyTorch can select optimized kernels depending on device, shape, and dtype.

After attention, the heads are transposed, made contiguous, reshaped back to `d_model`, and mixed through the output projection.

8.2 FeedForward

The FFN expands each token from `d_model` to `4*d_model`, applies GELU, and projects back. It never mixes positions directly; the identical network is applied to every position.

8.3 TransformerBlock

The code uses **pre-norm**:

```

x = x + attention(norm1(x))
x = x + ffn(norm2(x))

```

The two additions are residual connections. They preserve an identity path through the block and let sublayers learn corrections.

8.4 TinyTransformerLM

- Token and learned position embeddings are added.
- Several identical-in-structure but independently parameterized blocks are stacked.

- Final LayerNorm stabilizes the representation before vocabulary projection.
- Input and output weights are tied, so both refer to the same parameter.
- Flattening logits and targets lets cross-entropy treat every batch-position pair as one classification example.

8.5 Data and target alignment

If one sampled chunk is `trans`, the target is `ransf`. Thus each position predicts the following character. A real subword LM uses exactly the same alignment principle.

The example uses a contiguous 90/10 split. For document corpora, split by document or time where appropriate so nearly identical neighboring chunks do not leak across train and validation sets.

8.6 Training loop

Each iteration:

1. samples a batch;
2. computes next-token cross-entropy;
3. clears stale gradients;
4. performs backpropagation;
5. clips the total gradient norm;
6. updates parameters with AdamW.

Validation runs under `torch.no_grad()` and evaluation mode, which disables dropout and avoids gradient storage.

8.7 Generation

Only the final position's logits predict the next token. Temperature adjusts sharpness, top-k removes low-ranked choices, and `torch.multinomial` samples. The educational implementation recomputes the full current window every step; production models use a KV cache.

8.8 Hugging Face examples

The tokenizer returns token IDs and an attention mask. Dynamic padding pads only to the longest sequence in a batch. The pretrained classification head converts encoder states into two label logits. The fine-tuning example updates the pretrained parameters with a small learning rate; a production run should add an explicit metric function, reproducible seeds, error analysis, and task-appropriate data checks.

9. Training / Evaluation

9.1 Dataset preparation

For language-model pretraining:

- remove duplicates and near-duplicates to reduce memorization and benchmark contamination;
- identify document boundaries instead of blindly joining unrelated documents;
- filter corrupt, unsafe, private, or extremely low-quality text according to requirements;
- choose a language/domain mixture intentionally;

- tokenize once and store IDs efficiently;
- pack shorter documents to reduce padding while retaining boundary tokens;
- split by document, source, author, user, or time—not by randomly splitting overlapping chunks.

For supervised fine-tuning:

- define the label schema before annotation;
- inspect label balance and annotator agreement;
- keep exact and semantic duplicates in one split;
- ensure prompts at training and serving use compatible templates;
- mask prompt tokens from loss if the intended objective scores only the response;
- preserve a final test set until model selection is finished.

9.2 Train/validation/test split

Split	Purpose	Must not be used for
Train	gradient updates	final unbiased reporting
Validation	hyperparameters, early stopping, checkpoint choice	repeated claims of final generalization
Test	one-time final evaluation	model selection or prompt tuning

Use time-based splits for forecasting or systems whose future data differs from past data. Use group splits when records from the same user, patient, product, or document could leak identity-specific patterns.

9.3 Metrics

Choose metrics according to the task:

Task	Useful metrics	Important caution
Language modeling	NLL, perplexity, bits/token	tokenizer-dependent
Classification	accuracy, macro/micro F1, precision, recall, AUROC, calibration	accuracy hides imbalance
Translation	BLEU, chrF, COMET, human evaluation	lexical overlap is not full quality
Summarization	ROUGE, factuality, coverage, human preference	ROUGE may reward overlap over truth
Retrieval/embeddings	Recall@k, MRR, nDCG	evaluate realistic negatives
Generation/chat	task success, correctness, groundedness, safety, latency, preference	one aggregate score hides failure types
Production	p50/p95 latency, throughput, tokens/s, memory, cost/request	quality and systems metrics both matter

9.4 Overfitting and underfitting

Overfitting signs: training loss falls while validation loss rises; exact training phrases are memorized; calibration worsens; performance collapses on a new domain.

Mitigations include more diverse data, deduplication, dropout, weight decay, early stopping, smaller models, augmentation, label smoothing when appropriate, or freezing more pretrained layers.

Underfitting signs: both training and validation performance remain poor. Possible causes include insufficient capacity, too little training, a learning rate that is too small, excessive regularization, poor tokenization, noisy labels, or a mismatched pretrained model.

9.5 Important hyperparameters

Hyperparameter	Effect	Typical failure when wrong
learning rate	update size	divergence if high; slow/no learning if low
warm-up steps/ratio	stabilizes early optimization	early loss spikes without enough warm-up
batch size / tokens per batch	gradient noise and hardware utilization	OOM or unstable tiny batches
sequence length	visible context and quadratic attention cost	truncation or memory explosion
<code>d_model</code>	representation width	capacity/compute trade-off
number of layers	depth and iterative reasoning capacity	slow training, unstable optimization
number of heads	number and width of attention subspaces	head dimension too small or invalid shapes
<code>d_ff</code>	per-token nonlinear capacity	large parameter/compute cost
dropout	regularization	underfitting if too high
weight decay	parameter regularization	degraded fit if too strong
gradient clipping	protects against extreme gradients	too low suppresses useful updates
precision	speed and memory	numerical overflow/underflow if mishandled

9.6 Improving performance

Use this order of investigation:

1. Verify labels, splits, tokenization, masks, and metric implementation.
2. Establish a simple pretrained baseline.
3. Perform slice-based error analysis by length, class, domain, language, and input quality.
4. Improve data coverage or annotation where failures concentrate.
5. Tune learning rate, epochs, effective batch size, and sequence length.

6. Consider a better-matched pretrained checkpoint or parameter-efficient tuning.
7. Scale model or compute only after confirming data and evaluation are sound.

9.7 Reproducibility

Record code revision, model/tokenizer revision, preprocessing version, split identifiers, random seeds, package versions, hardware, precision, hyperparameters, and evaluation prompts. GPU training may still show small nondeterministic differences; report variation across seeds for important comparisons.

10. Complexity and Cost

10.1 Full self-attention complexity

For sequence length T and model width d :

- Q/K/V/output projections: approximately $O(Td^2)$;
- attention score and value products: approximately $O(T^2d)$;
- attention matrix memory: $O(T^2)$ per layer in the straightforward implementation;
- FFN compute: $O(Tdd_{ff})$.

The often-quoted **quadratic complexity** refers to the pairwise token interaction. For shorter sequences and large d , projections and FFNs may dominate actual compute.

Doubling context length approximately quadruples the score matrix size, while doubling model width has a different and often larger effect on projection/FFN computation.

10.2 Training memory

Training memory includes:

- model parameters;
- gradients;
- optimizer states, often two moment tensors for Adam;
- saved activations for backpropagation;
- temporary kernel workspaces.

Mixed precision, gradient accumulation, gradient checkpointing, sharding, efficient attention kernels, and optimizer-state partitioning reduce different parts of this cost. Gradient checkpointing saves activation memory by recomputing activations during backward, trading time for memory.

10.3 Inference phases

Prefill: Processes all prompt tokens, is highly parallel, and builds the KV cache.

Decode: Generates one token at a time. It is sequential across output positions and often limited by memory bandwidth because model weights and cached values must be read repeatedly.

Important serving metrics include time to first token, inter-token latency, tokens per second, throughput, p95 latency, batch utilization, and cost per request.

10.4 KV-cache memory

An approximate cache element count for a standard multi-head model is:

$$2 \times B \times L \times T \times h_{kv} \times d_{head},$$

where 2 represents keys and values. Multiply by bytes per element. Grouped-query attention reduces h_{kv} , often substantially reducing cache memory.

10.5 Hardware guidance

- The tiny educational model can run on CPU, though a GPU is faster.
- Fine-tuning small encoders is practical on a single consumer GPU with modest batches.
- Full fine-tuning of multi-billion-parameter models requires substantial accelerator memory or sharding.
- LoRA/QLoRA can make adaptation practical on much smaller hardware, but sequence length and activations can still cause OOM errors.
- Production inference depends on latency, concurrency, quantization, batching, context length, and output length—not parameter count alone.

11. Common Use Cases

1. **Text classification:** sentiment, intent, toxicity, topic, and ticket routing.
 2. **Token classification:** named-entity recognition, part-of-speech tagging, and sensitive-data detection.
 3. **Machine translation:** encoder-decoder mapping between languages.
 4. **Summarization:** abstractive summaries of documents, meetings, or conversations.
 5. **Question answering:** extractive answers, open-ended responses, and grounded RAG answers.
 6. **Semantic search:** Transformer encoders convert queries and documents into comparable vectors.
 7. **Reranking:** a cross-encoder scores query-document pairs with full interaction.
 8. **Code intelligence:** completion, explanation, repair, test generation, and code search.
 9. **Information extraction:** structured fields, relations, and event extraction.
 10. **Conversational assistants:** decoder-only generation after instruction and preference tuning.
 11. **Vision:** image classification, object detection, segmentation, and image generation.
 12. **Speech/audio:** transcription, speaker/language representations, audio generation.
 13. **Multimodal systems:** align and jointly reason over text, images, audio, and video.
 14. **Scientific modeling:** proteins, molecules, DNA, weather, and learned physical representations.
 15. **Time series and recommendations:** treat observations, events, or item histories as tokens.
-

12. Common Mistakes

12.1 Conceptual mistakes

- Saying attention “understands” meaning by itself; it computes learned weighted combinations.
- Claiming every head learns a specific linguistic rule. Interpretability is not guaranteed.
- Confusing embeddings with contextual hidden states.
- Saying Transformers have no order information; they require explicit or implicit positional mechanisms.
- Assuming encoder-only, encoder-decoder, and decoder-only models are interchangeable.
- Believing pretraining objective and architecture are the same thing.

- Treating lower perplexity as guaranteed better factuality or usefulness.
- Treating a larger context window as reliable use of every included token.

12.2 Masking and tensor bugs

- Applying the causal mask after softmax.
- Reversing mask meaning: some APIs use `True` for allowed positions, others for blocked positions.
- Forgetting the padding mask in encoder attention.
- Using causal masking for a bidirectional encoder by accident.
- Misaligning inputs and next-token targets.
- Computing loss on padding or instruction/prompt tokens unintentionally.
- Reshaping heads without the necessary transpose or `contiguous()` call.
- Forgetting to scale attention scores.
- Setting dropout nonzero in functional attention during evaluation.

12.3 Data and evaluation mistakes

- Randomly splitting overlapping chunks from the same document.
- Tuning prompts or checkpoints repeatedly on the test set.
- Comparing perplexity across different tokenizers as if token units were identical.
- Reporting only accuracy for severe class imbalance.
- Allowing benchmark answers or duplicates into pretraining/fine-tuning data.
- Truncating the label-bearing part of long examples.
- Evaluating generated text with only lexical-overlap metrics.
- Ignoring latency, memory, and cost when selecting a production model.

12.4 Fine-tuning mistakes

- Using a learning rate suitable for training from scratch on a pretrained model.
- Fine-tuning all parameters when a linear probe or LoRA baseline would answer the question.
- Changing tokenizer vocabulary without resizing the model embeddings.
- Adding tokens but failing to train their embeddings sufficiently.
- Forgetting `model.train()` during training or `model.eval()` during evaluation.
- Assuming gradient accumulation changes every behavior exactly like a physically larger batch; normalization and update schedules may differ.

12.5 Generation mistakes

- Using high temperature and then expecting deterministic factual answers.
- Combining restrictive top-k/top-p settings without checking for degraded output.
- Omitting stop conditions or maximum output length.
- Assuming beam search is best for open-ended chat; it can favor bland or repetitive sequences.
- Recomputing the full history in production instead of using a KV cache.
- Caching prompt states across users without strict isolation, risking privacy leakage.

13. Edge Cases / Limitations

13.1 Long contexts

Full attention is quadratic in sequence length. Even when a model technically accepts a long context, it may underuse middle information, retrieve distractors, or degrade when the position distribution differs from training.

13.2 Hallucination and calibration

A language model optimizes likely token sequences, not factual truth. It can state unsupported claims fluently. Retrieval, tools, citations, constrained generation, verification, and calibrated abstention help but do not eliminate the problem.

13.3 Rare, changing, and private knowledge

Parameter knowledge becomes stale and may not cover rare organizational facts. RAG or tools are often better for frequently changing data. Models may memorize sensitive strings from poorly governed datasets.

13.4 Tokenization failures

Unusual scripts, code, numbers, whitespace, long identifiers, and low-resource languages may fragment into many tokens. This increases cost and can hurt accuracy.

13.5 Numerical and implementation edge cases

- A query row with every key masked can produce invalid softmax behavior unless handled.
- Mixed-precision logits may overflow in naive attention implementations.
- Very large or tiny learning rates can cause divergence or stalled training.
- Empty strings, sequences containing only special tokens, or overlong examples need explicit policies.
- Left versus right padding can interact with position IDs and generation APIs.

13.6 Distribution shift

Performance may fail on new domains, languages, document layouts, adversarial phrasing, or time periods. Random benchmark splits often underestimate this risk.

13.7 Reasoning and reliability

Transformers may struggle with exact arithmetic, multi-step consistency, causal reasoning, compositional generalization, and maintaining constraints over long generations. Tool use and verifiers can improve reliability but introduce new failure modes.

13.8 Interpretability

Attention weights are not automatically faithful explanations. A high weight shows routing within one computation, not necessarily the causal reason for the final prediction.

13.9 Environmental and financial cost

Large-scale pretraining and serving consume significant accelerator time, energy, networking, and engineering resources. A smaller model, retrieval system, or classical method may be a better product choice.

14. Variations

Variation	What changes	When to use	Relevance
Encoder-only	bidirectional encoder stack	classification, NER, embeddings	essential for placements
Decoder-only	causal self-attention stack	generation, chat, code, general LLMs	essential
Encoder-decoder	source encoder plus cross-attentive decoder	translation, summarization, structured transduction	essential
Pre-norm Transformer	normalization before each sublayer	stable deep-model optimization	common advanced question
RoPE	rotates Q/K dimensions by position-dependent angles	relative position behavior in LLMs	important for LLM roles
ALiBi	adds distance-dependent bias to scores	simple long-context position bias	useful advanced topic
Relative attention	uses relative distances/representations	tasks where relative position matters	research/project value
Multi-query attention (MQA)	all query heads share one K/V head	low-memory, fast autoregressive serving	important for inference roles
Grouped-query attention (GQA)	groups of query heads share K/V heads	quality-efficiency compromise	widely used in modern LLMs
Sparse/local attention	permits only selected token pairs	long sequences with local/block structure	research and long-doc projects
Linear attention	replaces/approximates softmax attention using kernels or recurrence	very long sequences under specific assumptions	research-heavy; know trade-offs
FlashAttention	computes exact attention with IO-aware tiling without materializing the full score matrix	faster, memory-efficient GPU training/inference	highly relevant systems topic
Mixture of Experts (MoE)	routes each token through a subset of FFN experts	more parameter capacity at controlled per-token compute	advanced LLM/research topic
Vision Transformer (ViT)	converts image patches into tokens	image classification and representation learning	essential CV comparison
Swin Transformer	local shifted windows and hierarchy	dense vision tasks and higher-resolution images	useful CV project topic
DETR	object queries attend to image features	end-to-end object detection	common CV research topic

Variation	What changes	When to use	Relevance
Perceiver-style model	latent bottleneck cross-attends to large inputs	multimodal or very high-dimensional inputs	research-oriented
LoRA	trains low-rank updates to selected matrices	parameter-efficient adaptation	essential applied LLM topic
QLoRA	quantized base model plus LoRA training	fine-tuning with limited GPU memory	strong project/interview topic
Retrieval-augmented Transformer	adds retrieved external context	fresh, private, or source-grounded knowledge	essential AI engineering topic

14.1 BERT-style encoders

BERT uses bidirectional self-attention and masked-language-model pretraining. Use it when the output depends on understanding the complete input and free-form generation is not required. It remains a strong baseline for classification and token labeling.

14.2 GPT-style decoders

GPT-style models use causal language modeling. Their simple next-token interface supports open-ended generation and can express many tasks through text prompts. The causal constraint is ideal for generation but does not use future context within an input as directly as a bidirectional encoder.

14.3 T5/BART-style encoder-decoders

These models map an input text sequence to an output sequence. They are natural for translation, summarization, correction, and structured generation where the source and target have distinct roles.

14.4 Efficient attention is not one thing

Distinguish:

- **exact algorithm, better kernel:** FlashAttention changes memory access, not the mathematical result (up to numerical differences);
- **restricted pattern:** local/sparse attention omits some pairs;
- **approximation:** low-rank, kernel, or linear methods approximate/replace softmax attention;
- **cache optimization:** MQA/GQA reduces inference KV memory rather than the basic number of query positions.

This distinction is a strong interview answer because “efficient attention” can refer to different trade-offs.

15. Related Topics

15.1 RNN versus Transformer

Aspect	RNN/LSTM	Transformer
sequence processing	recurrent, step by step	parallel during training
long-range path	grows with token distance	direct attention connection
state	compressed recurrent hidden state	token-wise representations and attention
long-sequence cost	linear in length per layer	full attention quadratic in length
inference generation	sequential	also sequential for autoregressive decoding

RNNs can be attractive for streaming and strict memory constraints; Transformers dominate many large-scale representation and generation tasks.

15.2 CNN versus Vision Transformer

CNNs have built-in locality and translation-related inductive bias. ViTs treat image patches as tokens and learn global interactions with attention. CNNs can work well with less data; ViTs scale strongly with pretraining and flexible architectures. Hybrid models combine both.

15.3 Self-attention versus cross-attention

Self-attention exchanges information within one sequence. Cross-attention uses queries from one representation and keys/values from another. RAG generators may self-attend over a combined prompt, while encoder-decoder retrieval architectures may use explicit cross-attention.

15.4 LayerNorm versus BatchNorm

BatchNorm uses statistics across batch examples and is common in CNNs. LayerNorm normalizes feature dimensions within each token and behaves consistently at different batch sizes, which suits variable-length sequences and autoregressive inference.

15.5 Fine-tuning versus RAG

- **Fine-tuning** changes behavior or task skill by updating parameters.
- **RAG** supplies external evidence at inference time.

Use RAG for fresh/private knowledge and citations; fine-tune for format, style, task behavior, or domain adaptation. Many systems use both.

15.6 LoRA versus QLoRA

LoRA freezes the base model and learns low-rank parameter updates. QLoRA also stores the frozen base in low-bit quantized form to reduce memory. QLoRA improves accessibility but may add kernel, precision, and throughput trade-offs.

15.7 Transformer versus state-space models

State-space and recurrent alternatives can process long sequences with linear scaling and compact states. Attention offers flexible content-addressed access to previous tokens. Hybrid systems try to combine efficient recurrence with selective attention.

15.8 Transformers and RAG

A retriever finds candidate passages, an optional reranker improves ordering, and a Transformer generator conditions on the selected context. Failures can originate in retrieval, context construction, generation, or evaluation; diagnose these stages separately.

15.9 Transformers and reinforcement learning

Preference optimization can align a pretrained model with desired responses. RLHF commonly trains a reward model and optimizes a policy; direct preference methods optimize chosen/rejected response pairs more directly. The Transformer architecture remains the policy model, while the training objective changes.

15.10 Transformers and MLOps

Transformer systems require model/tokenizer versioning, experiment tracking, data lineage, evaluation suites, GPU-aware serving, quantization, batching, caching, safety monitoring, and drift analysis. Serving correctness includes using the exact tokenizer and chat template expected by the model.

16. Interview Questions

1. What problem did the Transformer solve compared with RNNs?

Answer: It removed recurrence as the main sequence-processing mechanism. Self-attention gives direct paths between distant tokens and allows all training positions to be processed in parallel. This improves accelerator utilization and long-range dependency modeling, although full attention introduces quadratic sequence cost.

2. Explain query, key, and value without using formulas.

Answer: A query states what the current token seeks, a key describes what another token can match on, and a value is the information that token contributes. Query-key similarity determines how much of each value is mixed into the output.

3. Why divide attention logits by $\sqrt{d_k}$?

Answer: With roughly unit-variance query/key components, their dot product variance grows with d_k . Large logits saturate softmax and produce small gradients. Scaling by $\sqrt{d_k}$ keeps variance and gradient behavior more stable.

4. What is the difference between self-attention and cross-attention?

Answer: In self-attention, Q, K, and V are derived from the same sequence. In cross-attention, queries come from one sequence—usually decoder states—and keys/values come from another—usually encoder outputs.

5. Why is positional encoding required?

Answer: Pure self-attention does not by itself distinguish token order. Position information makes sequences with identical tokens in different orders representationally different. It can be injected with absolute embeddings, relative biases, RoPE, or related methods.

6. What does multi-head attention add over one attention head?

Answer: It learns several projected attention spaces in parallel, allowing different interactions and feature subspaces to be represented. Head outputs are concatenated and mixed. It does not guarantee interpretable roles, and increasing head count narrows each head if model width is fixed.

7. Why does a Transformer need an FFN after attention?

Answer: Attention primarily routes and mixes information across tokens. The nonlinear FFN transforms features independently within each token. Together they provide cross-token communication and per-token nonlinear computation.

8. Why use LayerNorm instead of BatchNorm?

Answer: LayerNorm uses the features of each token and does not depend on batch statistics. It works with varying lengths, small batches, and autoregressive inference. BatchNorm's batch-dependent behavior is less natural for these settings.

9. Pre-norm or post-norm—which is preferred?

Answer: The original Transformer used post-norm. Many modern deep Transformers use pre-norm because the residual stream provides a cleaner gradient path and training is generally more stable. Exact choices vary, and output normalization or alternative norms may also be used.

10. How can decoder-only training be parallel if generation is sequential?

Answer: During training, the whole known target sequence is available. A causal mask prevents each position from seeing future tokens, so all next-token losses can be computed in one forward pass. At inference, each unknown next token must be generated before the following step can run.

11. What is teacher forcing, and what mismatch does it cause?

Answer: Teacher forcing feeds ground-truth previous target tokens to the decoder during training. At inference the model consumes its own predictions, so errors can change future inputs and compound. This difference is often called exposure bias.

12. What are the time and memory complexities of full self-attention?

Answer: Pairwise score computation is $O(T^2d)$, and straightforward attention-matrix memory is $O(T^2)$. Linear projections add $O(Td^2)$, and the FFN adds $O(Tdd_{ff})$. Which term dominates depends on sequence length and model width.

13. What is the KV cache?

Answer: It stores keys and values from previous tokens at every decoder layer. New tokens then reuse those states instead of recomputing the full prefix. It speeds autoregressive decoding but consumes memory growing linearly with context length and does not eliminate sequential generation.

14. Compare encoder-only, encoder-decoder, and decoder-only architectures.

Answer: Encoder-only models use bidirectional context and suit understanding tasks. Encoder-decoder models encode a source and generate a separate target through causal self-attention plus cross-attention.

Decoder-only models causally predict the next token and dominate general-purpose generation.

15. What is the difference between padding and causal masks?

Answer: A padding mask blocks artificial padding positions. A causal mask blocks future positions. A decoder batch may need both. Their tensor shapes and Boolean semantics depend on the library API.

16. What is perplexity, and when is it misleading?

Answer: Perplexity is the exponential of mean token negative log-likelihood. Lower is better on the same data and tokenization. It is misleading across tokenizers or domains and does not directly measure factuality, safety, instruction following, or end-task utility.

17. What is weight tying?

Answer: The input token embedding matrix and output vocabulary projection share parameters, usually by using the transpose-compatible same weight. This reduces parameter count and can improve representation learning.

18. Explain FlashAttention in an interview.

Answer: FlashAttention computes exact softmax attention using IO-aware tiling so intermediate score/probability matrices need not be fully materialized in slow high-bandwidth memory. It reduces memory traffic and activation memory; it is a kernel/algorithm optimization, not an approximation of attention.

19. MHA versus MQA versus GQA?

Answer: Standard MHA gives every query head its own K/V heads. MQA shares one K and one V head across all query heads, reducing cache memory and bandwidth. GQA uses several K/V groups, offering a quality-efficiency compromise.

20. Why can attention weights be a poor explanation?

Answer: Model output also depends on value vectors, residual paths, FFNs, later layers, and nonlinear interactions. A large attention weight does not prove that token causally determined the prediction. Faithful explanation needs interventions or other causal analyses.

21. How would you debug a Transformer whose loss does not decrease?

Answer: First overfit one tiny batch. Verify target shift, masks, ignored padding, vocabulary range, logits shape, and train mode. Then inspect learning rate, gradients, NaNs, initialization, frozen parameters, precision scaling, optimizer order, and whether labels contain signal.

22. How would you handle documents longer than the context window?

Answer: Depending on the task: chunk with overlap, retrieve relevant chunks, summarize hierarchically, use a long-context/sparse model, or encode chunks and aggregate representations. Preserve document-level splits and evaluate whether information crossing chunk boundaries is lost.

23. When should you fine-tune instead of use RAG?

Answer: Fine-tune to change stable behavior, output structure, task execution, or domain representation. Use RAG for frequently changing, private, extensive, or citable knowledge. If both behavior and current knowledge matter, combine them.

24. Why might validation loss improve while generated answers get worse?

Answer: Token likelihood may not align with the desired task metric; the validation distribution or prompt format may differ; decoding settings may change; the model may become more confident but less factual; or aggregate loss may hide degradation in important slices. Use task-specific evaluation and error analysis.

25. How would you reduce Transformer inference cost?

Answer: Choose a smaller or distilled model, quantize weights/cache, batch requests, use optimized kernels, cache reusable prefixes, use KV caching and GQA/MQA, shorten prompts/outputs, route easy requests to smaller models, or distill the task. Measure quality, p95 latency, throughput, and cost together.

26. Does attention have to be $O(T^2)$?

Answer: Dense all-pairs attention does. Local/sparse patterns reduce the number of pairs, linear-attention methods reformulate or approximate the operation, and recurrent/state-space alternatives avoid all-pairs attention. FlashAttention reduces memory traffic and stored intermediates but retains dense attention's asymptotic arithmetic.

27. What is RoPE at a high level?

Answer: Rotary positional embedding applies position-dependent rotations to query and key vector pairs. Their dot product then naturally depends on relative position differences. It is widely used in decoder-only LLMs and can be extended or scaled for longer contexts, though extrapolation is not automatically perfect.

28. What is the strongest simple baseline for a new NLP classification task?

Answer: Start with a correctly split dataset, a majority/linear baseline, then fine-tune a small pretrained encoder such as a BERT-family model. This establishes whether a larger generative model is actually necessary.

17. Practice Tasks

Task 1: Small coding task—attention by hand

Implement scaled dot-product attention using only basic PyTorch tensor operations. Add a causal mask, and compare its output with `torch.nn.functional.scaled_dot_product_attention` with dropout disabled.

Checks: output shape, row sums after softmax, zero probability on forbidden positions, and numerical closeness to PyTorch.

Task 2: Dataset-based project—sentiment classification

Fine-tune a small pretrained encoder on SST-2 or IMDb.

Requirements: document-safe split, dynamic padding, accuracy plus F1, confusion matrix, error slices by text length, and latency comparison between CPU and GPU.

Task 3: Experiment—position and context

Train tiny language models with:

1. no position embeddings;
2. learned absolute embeddings;
3. sinusoidal embeddings.

Compare validation loss on seen lengths and longer lengths. Explain why no-position performance changes and whether extrapolation claims are supported.

Task 4: Debugging task—find the leakage

Create an intentionally broken language model that uses no causal mask. Observe unusually low training/validation loss, then inspect attention maps and correct the mask. Explain why the model could copy the target token through its current/future representation.

Task 5: Analysis task—head behavior

Visualize attention from several layers and heads for sentences containing pronouns or long-distance dependencies. Treat the plots as hypotheses, then use token masking or activation interventions to test whether visually strong weights are causally important.

Task 6: Extension—padding-aware classifier

Extend the code with an encoder mode, batches of variable-length sequences, a padding mask, and masked mean pooling. Verify that adding more right-padding does not change predictions beyond numerical tolerance.

Task 7: Systems experiment—sequence scaling

Benchmark time and peak GPU memory at sequence lengths 128, 256, 512, and 1024. Plot scaling, explain when quadratic attention becomes visible, and compare standard attention with an optimized scaled-dot-product kernel.

Task 8: Generation experiment

For a small pretrained causal LM, compare greedy, beam, temperature, top-k, and top-p decoding on fixed prompts. Measure diversity, repetition, task correctness, and latency. Hold random seeds fixed where sampling is used.

Task 9: Fine-tuning versus retrieval

Build a small QA corpus whose facts change between two versions. Compare prompting, fine-tuning, and RAG. Evaluate answer accuracy, citation correctness, update cost, and failure behavior when retrieval returns irrelevant text.

Task 10: Interview whiteboard exercise

Given $B = 8$, $T = 512$, $d_{model} = 768$, $h = 12$, derive Q/K/V shapes, per-head dimension, attention-score shape, and approximate score-matrix element count. Then explain how each changes when T doubles.

18. Project Ideas

Project 1: Evidence-grounded placement assistant

What it does: Retrieves answers from verified placement policies, job descriptions, and preparation notes, then generates an answer with passage-level citations and an abstention when evidence is weak.

Tech stack: Python, PyTorch/Hugging Face, sentence-transformer embeddings, FAISS or a vector database, a reranker, FastAPI, Docker, and an evaluation script.

Dataset suggestion: Public company career FAQs plus a curated local corpus of role descriptions and interview notes. Create answerable, unanswerable, and conflicting-evidence test questions.

Resume value: Demonstrates Transformers, embeddings, retrieval, reranking, grounding, API deployment, evaluation, and practical hallucination control. Report Recall@k, answer accuracy, citation precision, p95 latency, and cost/query.

Project 2: Long-document issue and action-item extractor

What it does: Processes meeting transcripts or support conversations, identifies decisions, owners, deadlines, and unresolved issues, and produces validated JSON.

Tech stack: Hugging Face encoder or encoder-decoder model, PyTorch, chunking plus aggregation, Pydantic validation, FastAPI, MLflow, and a small review UI.

Dataset suggestion: AMI Meeting Corpus, QMSum, SAMSum, or a carefully anonymized custom dataset with span and structured labels.

Resume value: Shows long-context design, structured generation, schema validation, token-level versus sequence-level evaluation, error analysis, and deployment. Report field-level precision/recall/F1 and exact-record accuracy.

Project 3: Efficient Transformer inference benchmark

What it does: Compares small causal language models across precision, quantization, batch size, prompt length, output length, and decoding strategies.

Tech stack: PyTorch, Hugging Face Transformers, an optimized inference engine where available, GPU monitoring, Pandas, and a dashboard or reproducible notebook.

Dataset suggestion: A fixed prompt suite containing summarization, extraction, QA, and code tasks; use public evaluation subsets with license-compatible prompts.

Resume value: Demonstrates systems-level understanding. Report time to first token, inter-token latency, tokens/s, peak memory, quality metrics, and cost-normalized performance—not just screenshots of generated text.

19. Quick Revision

Key idea

A Transformer repeatedly combines **attention**, which routes information between tokens, with a **feed-forward network**, which nonlinearly transforms each token. Residual connections, normalization, masking, and positional information make the stack trainable and sequence-aware.

Main formula

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^{\top}}{\sqrt{d_k}} + M \right) V$$

When to use

- Use encoder-only models for representation, classification, and token labeling.
- Use encoder-decoder models for explicit source-to-target generation.
- Use decoder-only models for next-token generation, prompting, and general LLM behavior.
- Use retrieval/tools when knowledge must be current, private, or verifiable.
- Consider simpler models when data, latency, interpretability, or compute constraints dominate.

Important metrics

- LM: cross-entropy/perplexity plus downstream behavior metrics.
- Classification: precision, recall, F1, AUROC, calibration, and slice metrics.
- Generation: correctness, groundedness, task success, preference, and safety.
- Serving: time to first token, inter-token latency, throughput, memory, and cost.

Common traps

- incorrect causal or padding mask;
- target shift off by one;
- data leakage through overlapping documents;
- comparing perplexity across tokenizers;
- confusing decoding changes with model learning;
- claiming attention weights are explanations;
- ignoring KV-cache and long-context memory;
- using a large generative model when a small encoder or linear baseline suffices.

Interview one-liner

A Transformer is a stack of attention and position-wise feed-forward blocks that uses learned content-based routing, positional information, residual connections, and normalization to model sequences efficiently and at scale.

20. Final Cheat Sheet

Item	Transformer answer
Definition	Neural architecture that models token interactions primarily through attention rather than recurrence
Input	token IDs/patches/features, position information, and masks

Item	Transformer answer
Output	contextual token states, task logits, embeddings, or next-token probabilities
Main steps	tokenize → embed + position → Q/K/V → masked scaled attention → combine heads → residual/norm → FFN → repeat → task head
Core equation	$\text{softmax}(QK^\top / \sqrt{d_k} + M)V$
Architectures	encoder-only, encoder-decoder, decoder-only
Main loss	token or label cross-entropy; objective depends on task
Key hyperparameters	layers, d_{model} , heads, d_{ff} , context length, dropout, learning rate, warm-up, batch/token budget
LM metrics	NLL, perplexity, task success, factuality/groundedness
Classification metrics	precision, recall, F1, accuracy, AUROC, calibration
Systems metrics	TTFT, inter-token latency, tokens/s, throughput, memory, cost/request
Training cost	projections/FFN plus dense attention; activation and optimizer memory are major costs
Attention complexity	$O(T^2d)$ arithmetic for pairwise interaction and straightforward $O(T^2)$ score memory
Strengths	direct long-range interactions, parallel training, transfer learning, scaling, multimodal flexibility
Weaknesses	long-context cost, data/compute needs, hallucination, tokenization sensitivity, difficult interpretation
Inference optimization	KV cache, batching, quantization, optimized kernels, MQA/GQA, shorter prompts/outputs
Best use cases	language understanding/generation, vision, speech, multimodal systems, retrieval/reranking, structured sequence modeling
Biggest implementation trap	incorrect masking or next-token target alignment
Best debugging check	overfit one tiny batch, then verify masks, shapes, label shift, gradients, and evaluation mode

Thirty-second recall

1. Embeddings turn token IDs into vectors; position mechanisms encode order.
2. Queries match keys; normalized scores mix values.
3. Multiple heads operate in learned subspaces.
4. Attention mixes tokens; FFNs transform each token.
5. Residual connections and normalization stabilize deep stacks.
6. Encoders see both directions; causal decoders cannot see future tokens.
7. Cross-entropy trains token or task predictions with backpropagation.

8. Training is parallel across known positions; autoregressive generation is sequential.
9. Full attention is quadratic in sequence length; KV caching accelerates decoding but costs memory.
10. Evaluate the actual task, data slices, latency, memory, and cost—not loss alone.